

w3L12: w3L1: Language Modelling: Advanced Smoothing Models

Advanced smoothing algorithms

Some Examples

- Good-Turing
- Kneser-Ney

Advanced smoothing algorithms

Some Examples

- Good-Turing
- Kneser-Ney

Good-Turing: Basic Intuition

Not seen word ->

Once -> zero

Twice are used for once

Use the count of things we have seen once

- to help estimate the count of things we have **never seen**

Nc: Frequency of frequency c

Example Sentences

<s>I am here </s>

<s>who am I </s>

<s>I would like </s>

Computing N_c

I	3
am	2
here	1
who	1
would	1
like	1

n1 how many words occur once, n2 - twice, n4 - 4times

$$N_1 = 4$$

$$N_2 = 1$$

$$N_3 = 1$$

Good Turing Estimation

Idea

- Reallocate the probability mass of n-grams that occur $r+1$ times in the training data to the n-grams that occur r times
- In particular, reallocate the probability mass of n-grams that were seen once to the n-grams that were never seen

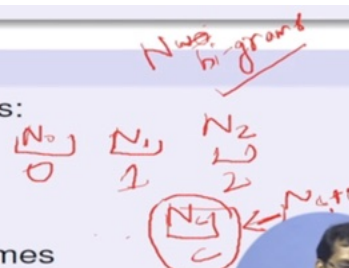
Once $\rightarrow$ zero : for t-times

Adjusted count

For each count c , an adjusted count c^* is computed as:

$$c^* = \frac{(c+1)N_{c+1}}{N_c}$$

where N_c is the number of n -grams seen exactly c times



N_0 zero time, N_1 - once, N_c - c times

N words total - bigrams, N_{c+1} diff bigrams

$N_{c+1} (C+1) / N \rightarrow$ distributing to words with c times divide by N_c

Effective count $\rightarrow c^*/N$

Good Turing Estimation

Good Turing Smoothing - n-grams not seen even once

$$P_{GT}^*(\text{things with frequency } c) = \frac{c^*}{N}$$

$$c^* = \frac{(c+1)N_{c+1}}{N_c}$$



How much will non-seen n-gram get? Divide by n-grams which are not seen in train data

What if $c = 0$

$$P_{GT}^*(\text{things with frequency } c) = \frac{N_1}{N} \text{ where } N \text{ denotes the total number of bigrams that actually occur in training}$$

Ans: Bigram/ Number of n-grams didn't occur

Complications

What about words with high frequency?

- For small c , $N_c > N_{c+1}$
- For large c , too jumpy

Simple Good-Turing

Replace empirical N_k with a best-fit power law once counts get unreliable

Adjust using power law

They do for initial few values of k

Good-Turing numbers: Example APN Academic Press News

22 million words of AP Neswire

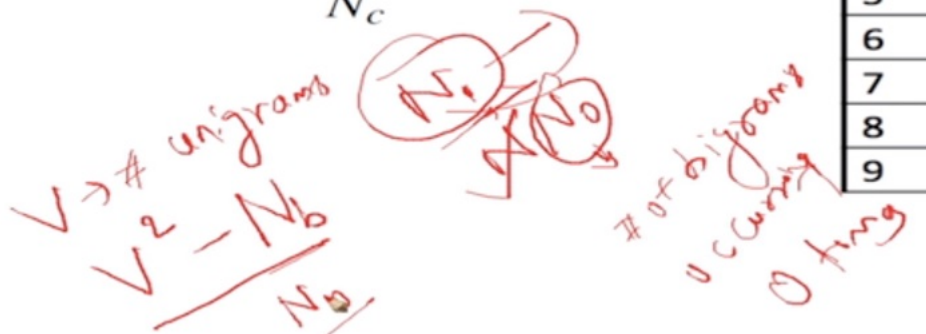
$$c^* = \frac{(c+1)N_{c+1}}{N_c}$$

Count c	Good Turing c^*
0	.0000270
1	0.446
2	1.26
3	2.24
4	3.24
5	4.22
6	5.19
7	6.21
8	7.24
9	8.25

No. of bigrams didn't occur in corpus N-zero = $V^2 - N_b$ (actual bigrams - 300k)
N1 is unique diagram

22 million words of AP Neswire

$$c^* = \frac{(c+1)N_{c+1}}{N_c}$$



Absolute discounting method: value = prev value minus 0.75

It looks like $c^* = c - 0.75$

Absolute Discounting Interpolation

Why don't we just subtract 0.75 (or some d)?

$$P_{AbsoluteDiscounting}(w_i|w_{i-1}) = \frac{c(w_{i-1}, w_i) - d}{c(w_{i-1})} + \lambda(w_{i-1})P(w_i)$$

Normalization constant will adapt to 1

$$P_{AbsoluteDiscounting}(w_i|w_{i-1}) = \frac{c(w_{i-1}, w_i) - d}{c(w_{i-1})} + \lambda(w_{i-1})P(w_i)$$

$$\sum_{w_i} P_{Abs. D.}(w_i|w_{i-1}) = 1$$

We may keep some more values of d for counts 1 and 2

But can we do better than using the regular unigram correct?

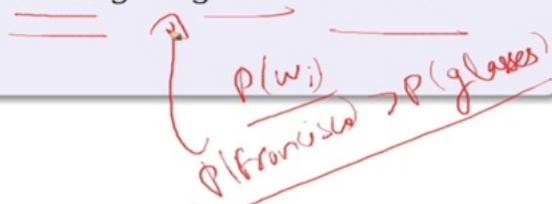
Kneser-Ney Smoothing

Intuition

Shannon game: I can't see without my reading ...: glasses/Francisco?

Intuition

- Shannon game: *I can't see without my reading ...: glasses/Francisco?*



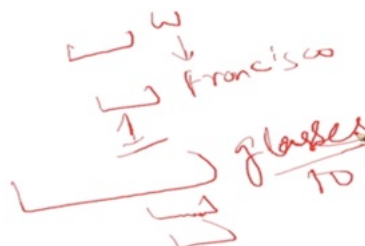
If it is US corpus, will fill with Francisco as “Francisco” more common than “glasses”

-

Intuition: But “Francisco” mostly follows “San”

- What are the other words it comes with
- Glasses is better choice as it occurs with many words

Francisco occurs after san so glasses is good choice



$P(w)$: “How likely is w ?”

Instead, $P_{\text{continuation}}(w)$: “How likely is w to appear as a novel continuation?”

- For each word, count the number of bigram types it completes
- Every bigram type was a novel continuation the first time it was seen

$$P_{\text{continuation}}(w) \propto |\{w_{i-1} : c(w_{i-1}, w) > 0\}|$$

Occurring first time is Novel contribution

Or occurs as last word

How do I normalize - prop - no. of bigrams it is completing

Total No. of bigram TYPES (not TOKENS)

Normalize by all possible bigrams type

How many times does w appear as a novel continuation?

$$P_{\text{continuation}}(w) \propto |\{w_{i-1} : c(w_{i-1}, w) > 0\}|$$

Normalized by the total number of word bigram types

$$|\{(w_{j-1}, w_j) : c(w_{j-1}, w_j) > 0\}|$$

$$P_{\text{continuation}}(w) = \frac{|\{w_{i-1} : c(w_{i-1}, w) > 0\}|}{|\{(w_{j-1}, w_j) : c(w_{j-1}, w_j) > 0\}|}$$

A frequent word (Francisco) occurring in only one context (San) will have a low continuation probability

$$P_{KN}(w_i | w_{i-1}) = \frac{\max(c(w_{i-1}, w_i) - d, 0)}{c(w_{i-1})} + \lambda(w_{i-1}) P_{\text{continuation}}(w_i)$$

$$P_{KN}(w_i | w_{i-1}) = \frac{\max(c(w_{i-1}, w_i) - d, 0)}{c(w_{i-1})} + \lambda(w_{i-1}) P_{\text{continuation}}(w_i)$$

Handwritten notes:
 $\lambda < d$
 $\sum_{w_i \in V} P_{KN}(w_i | w_{i-1}) = 1$

Instead of $P(w_i)$ -> replaced $P_{\text{continuation}}(w_i)$

ToDo: Exercise find λ . Answer is:

λ is a normalizing constant

$$\lambda(w_{i-1}) = \frac{d}{c(w_{i-1})} |\{w : c(w_{i-1}, w) > 0\}|$$

Why can't we combine models:

Use MLE

Model Combination

As N increases

- The power (expressiveness) of an N-gram model increases (when saw larger n-gram, lower perplexity)
- But the ability to estimate accurate parameters from sparse data decreases (i.e. the smoothing problem gets worse).

Problem with larger n : data becomes sparse

A general approach is to combine the results of multiple N-gram models. (take adv of large n-gram model when you have data)

Backoff and Interpolation

It might help to use less context

- when you haven't learned much about larger contexts

Backoff

- use trigram if you have good evidence
- otherwise bigram, otherwise unigram

Interpolation

mix unigram, bigram, trigram

Backoff

Estimating $P(w_i|w_{i-2}w_{i-1})$

- If we do not have counts to compute $P(w_i|w_{i-2}w_{i-1})$ estimate this using the bigram probability $P(w_i|w_{i-1})$

Handwritten notes:
 $C(w_{i-2}w_{i-1}w_i) = 0$
 $C(w_{i-1}w_i) = 0$
 $P(w_i)$
 $P(w_{i-1})$
 $P(w_i|w_{i-1})$

Estimating $P(w_i|w_{i-2}w_{i-1})$

- If we do not have counts to compute $P(w_i|w_{i-2}w_{i-1})$ estimate this using the bigram probability $P(w_i|w_{i-1})$
- If we do not have counts to compute $P(w_i|w_{i-1})$, estimate this using the unigram probability $P(w_i)$

L12: 30min:

P-hat is similar to MLE - prob mass is stolen and can be given to other

With MLE estimate 3/5, 2/5

Estimating $P(w_i|w_{i-2}w_{i-1})$

- If we do not have counts to compute $P(w_i|w_{i-2}w_{i-1})$ estimate this using the bigram probability $P(w_i|w_{i-1})$
- If we do not have counts to compute $P(w_i|w_{i-1})$, estimate this using the unigram probability $P(w_i)$

$$P_{bo}(w_i|w_{i-2}w_{i-1}) =$$

- $\hat{P}(w_i|w_{i-2}w_{i-1})$, if $c(w_{i-2}w_{i-1}w_i) > 0$
- $\lambda(w_{i-1}w_{i-2})P_{bo}(w_i|w_{i-1})$, otherwise

$$\text{where } P_{bo}(w_i|w_{i-1}) =$$

- $\hat{P}(w_i|w_{i-1})$ if $c(w_{i-1}w_i) > 0$
- $\lambda(w_{i-1})\hat{P}(w_i)$, otherwise

Handwritten notes: w_1, w_2, w_3 with arrows pointing to $P(w_3|w_2)$ and $P(w_3|w_1)$. A red circle highlights $P(w_3|w_2)$ with the value 2/5 written next to it.



Pb0 is recursive definition -you backoff

$$P_{bo}(w_i|w_{i-2}w_{i-1}) =$$

- $\hat{P}(w_i|w_{i-2}w_{i-1})$, if $c(w_{i-2}w_{i-1}w_i) > 0$
- $\lambda(w_{i-1}w_{i-2})P_{bo}(w_i|w_{i-1})$, otherwise

$$\text{where } P_{bo}(w_i|w_{i-1}) =$$

- $\hat{P}(w_i|w_{i-1})$ if $c(w_{i-1}w_i) > 0$
- $\lambda(w_{i-1})\hat{P}(w_i)$, otherwise

Example Problem: In a corpus, suppose there are 4 words, a, b, c, and d. You are provided with the following counts.

a occurs after ab 4 times

n-gram	count	n-gram	count	n-gram	count
<u>aba</u>	4	<u>ba</u>	5	a	8
<u>abb</u>	0	<u>bb</u>	3	b	9
abc	0	<u>bc</u>	0	c	8
abd	0	bd	0	d	7

Find backoff prob distribution

Use the recursive definition of backoff smoothing to obtain the probability distribution, $P_{\text{backoff}}(w_n | w_{n-2}w_{n-1})$, where $w_{n-1} = b$ and $w_{n-2} = a$.

Also assume that $P(x) = P(x) - 1/8$.

$P_{bo}(w | ab)$

$$P_{bo}(w | ab) \begin{cases} a \rightarrow 4 \rightarrow \hat{p}(a|ab) \\ b \rightarrow 0 \rightarrow \lambda_{ab} \cdot P_{bo}(b|b) \\ c \rightarrow 0 \rightarrow \lambda_{ab} \cdot P_{bo}(c|b) \\ d \rightarrow 0 \rightarrow \lambda_{ab} \cdot P_{bo}(d|b) \end{cases}$$

$$P_{bo}(w | b) \begin{cases} a \rightarrow 5 \rightarrow \hat{p}(a|b) = 5/8 - 1/8 = 4/8 \\ b \rightarrow 3 \rightarrow \hat{p}(b|b) = 3/8 - 1/8 = 2/8 \\ c \rightarrow 0 \rightarrow \lambda_b \cdot \hat{p}(c) = \lambda_b \cdot [P(c) - 1/8] \\ d \rightarrow 0 \rightarrow \lambda_b \cdot \hat{p}(d) = \lambda_b \cdot [P(d) - 1/8] \end{cases}$$

$$= \lambda_b \cdot \left[\frac{7}{32} - \frac{1}{8} \right] = \lambda_b \cdot \frac{3}{32}$$

Find lambda-b

$$P_{b0}(w|b) = \lambda_b \left[\frac{1}{8} + \frac{3}{32} \right] + \frac{4}{8} + \frac{2}{8} = 1$$

$$\Rightarrow \lambda_b \left[\frac{7}{32} \right] = \frac{2}{8}$$

$$\lambda_b = \frac{8}{14}$$

$$- c \rightarrow 0 \rightarrow \lambda_b \cdot \hat{P}(c) = \lambda_b \cdot \left[P(c) - \frac{1}{8} \right]$$

$$- d \rightarrow 0 \rightarrow \lambda_b \cdot \hat{P}(d) = \lambda_b \cdot \left[P(d) - \frac{8}{32} \right]$$

from lambda-b, find prob of b and c to get pbo(w|b)

$$P_{b0}(w|ab) = \begin{cases} b \rightarrow 0 \rightarrow \lambda_{ab} \cdot P_{b0}(b|b) \leftarrow \\ c \rightarrow 0 \rightarrow \lambda_{ab} \cdot P_{b0}(c|b) \leftarrow \\ d \rightarrow 0 \rightarrow \lambda_{ab} \cdot P_{b0}(d|b) \leftarrow \end{cases}$$

Put in Pb0 (on right of eq.) and then find Pbo (w | ab)

Linear Interpolation

Simple Interpolation - from diff models (tri, bi, uni gram models)

$$\tilde{P}(w_n | w_{n-1} w_{n-2}) = \lambda_1 P(w_n | w_{n-1} w_{n-2}) + \lambda_2 P(w_n | w_{n-1}) + \lambda_3 P(w_n)$$

They will add to 1: $\sum_i \lambda_i = 1$

Lambdas conditional on context - now lambda is dependent on context

$$\tilde{P}(w_n | w_{n-1} w_{n-2}) = \lambda_1(w_{n-2}, w_{n-1}) P(w_n | w_{n-1} w_{n-2})$$

$$+ \lambda_2(w_{n-2}, w_{n-1}) P(w_n | w_{n-1}) + \lambda_3(w_{n-2}, w_{n-1}) P(w_n)$$

Setting the lambda values

Use a held-out corpus

Choose Is to maximize the probability of held-out data:

Find the N-gram probabilities on the training data

Search for Is that give the largest probability to held-out data

w3L13: w3L2: Computational Morphology

Morphology

Morphology studies the internal structure of words, how words are built up from smaller meaningful units called morphemes

dogs

- 2 morphemes, 'dog' and 's'
- 's' is a plural marker on nouns

unladylike

3 morphemes

- un- 'not'
- lady 'well-behaved woman'
- -like 'having the characteristic of'

Allomorphs

Variants of the same morpheme, but cannot be replaced by one another

Example

- opposite: un-happy, in-comprehensible, im-possible, ir-rational

Different ways to make opposite

Bound and Free Morphemes

unhappy: un is not a word in itself, happy is word in itself

Bound

Cannot appear as a word by itself.

-s (dog-s), -ly (quick-ly), -ed (walk-ed),

Free

Can appear as a word by itself; often can combine with other morphemes too.

house (house-s), walk (walk-ed), of, the, or

Of, in are free morphin

Stems and Affixes (Important)

Stems and Affixes

- Stems (roots): The **core** meaning bearing units (boy in boys)
- Affixes: Bits and pieces adhering to stems to change their meanings and grammatical functions (s at the end to make it plural)

Mostly, stems are free morphemes and affixes are bound morphemes

Types of affixes

- **Prefix:** un-, anti-, etc (a-, ati-, pra- etc.)
 - un-happy, pre-existing
- **Suffix:** -ity, -ation, etc (-taa, -ke, -ka etc.)
 - talk-ing, quick-ly
- **Infix:** 'n' in 'vindati' (he knows), as contrasted with vid (to know).
 - Philippines: basa 'read' ! b-um-asa 'read'
 - English: abso-bloody-lutely (emphasis) - (Not common)
- **Circumfixes** - precedes and follows the stem
 - Dutch: berg 'mountain', ge-berg-te 'mountains'

Content and functional morphemes

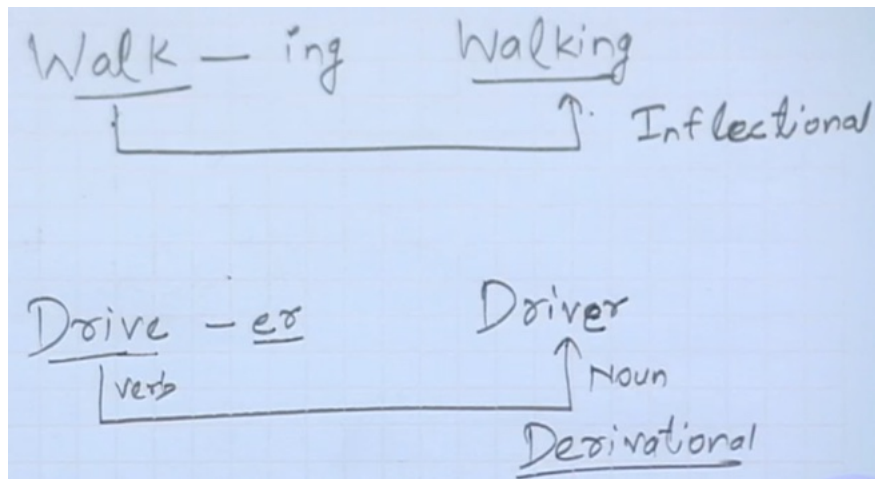
Content morphemes

- Carry some **semantic** content (meaning)
- car, -able, un-

Functional morphemes

- Provide **grammatical** information
- -s (plural), -s (3rd singular)

Inflectional and Derivational Morphology



Two different kind of relationship among words

Inflectional morphology

- Grammatical: number, tense, case, gender
- Creates new forms of the same word: bring, brought, brings, bringing

Derivational morphology

- Creates new words by changing part-of-speech: logic, logical, illogical, illogicality, logician
- Fairly systematic but some derivations are missing: sincere - sincerity, scarce - scarcity, curious - curiosity, fierce - fierceness

Morphological processes

Concatenation

Adding continuous affixes - the most common process:

- hope+less, un+happy, anti+capital+ist+s

Often, there are phonological/graphemic changes on morpheme boundaries:

- book + s [s], shoe + s [z]
- happy +er ! happier

Changes happen at the morpheme boundary, how they are pronounced

Grapheme - how they are combined

- book + s [s], shoe + s [z]

Phoneme -

- happy +er ! happier

Reduplication: part of the word or the entire word is doubled

- Nama: 'go' (look), 'go-go' (examine with attention)
- Tagalog: 'basa' (read), 'ba-basa' (will read)
- Sanskrit: 'pac' (cook), 'papāca' (perfect form, cooked)
- Phrasal reduplication (Telugu): *pillavāḍu naḍustū naḍustū paḍi pōyāḍu*
(The child fell down while walking)

Suppletion

- 'irregular' relation between the words
- go - went, good - better

Morpheme internal changes

The word changes internally

- sing - sang - sung, man - men, goose - geese

Word Formation

Compounding Words formed by combining two or more words Example in English:	• Adj + Adj → Adj: bitter-sweet • N + N → N: rain-bow • V + N → V: pick-pocket • P + V → V: over-do
---	---

Particular to languages

room-temperature: Hindi translation?

Word Formation

Acronyms

- laser: Light Amplification by Simulated Emission of Radiation

Blending

Parts of two different words are combined

- breakfast + lunch ! brunch
- smoke + fog ! smog
- motor + hotel ! motel

Clipping

Longer words are shortened

- doctor, laboratory, advertisement, dormitory, examination, bicycle, refrigerator

22min: Processing morphology

Lemmatization: word ! lemma

saw -> {fsee, saw}

Morphological analysis : word -> setOf(lemma +tag)

saw -> { <see, verb.past>, < saw, noun.sg> }

Tagging: word -> tag, considers context

Peter saw her -> { <see, verb.past> }

Morpheme segmentation: de-nation-al-iz-ation

Generation: see + verb.past -> saw

What are the applications?

Text-to-speech synthesis:

- lead: verb or noun? (diff pronunciation if it is leed or led)
- read: present or past?

Search and information retrieval (reduce search space)

Machine translation, grammar correction

Morphological Analysis - we want right column output from a given word

Input	Morphological Parsed Output
cats	cat +N +PL
cat	cat +N +SG
cities	city +N +PL
geese	goose +N +PL
goose	(goose +N +SG) or (goose +V)
gooses	goose +V +3SG
merging	merge +V +PRES-PART
caught	(catch +V +PAST-PART) or (catch +V +PAST)

Goal

To take input forms like those in the first column and produce output forms like those in the second column.

Output contains stem and additional information; +N for noun, +SG for singular, +PL for plural, +V for verb etc.

Issues involved in processing of morphology

boy -> boys

fly -> flys ! flies (y -> i rule)

Toiling -> toil

Duckling -> duckl? (duckl not correct)

Getter -> get + er

Doer -> do + er

Beer -> be + er?

Knowledge Required

Knowledge of stems or roots

- Duck is a possible root, not duckl.
- We need a dictionary (lexicon)

Morphotactics

- Which class of morphemes **follow** other classes of morphemes inside the word?
- Ex: plural morpheme follows the noun

Only some endings go on some words

- Do+er: ok
- Be+er: not so

Spelling change rules

Adjust the surface form using spelling change rules

- Get + er ! getter

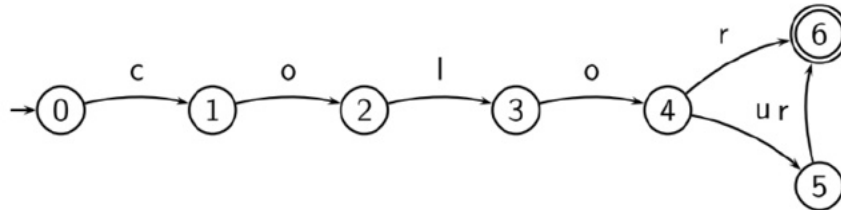
Why can't this be put in a big lexicon?

- English: just 317,477 forms from 90,196 lexical entries, a ratio of 3.5:1
 - For English is it is easier
- Sanskrit: 11 million forms from a lexicon of 170,000 entries, a ratio of 64.7:1
 - Ratio is 64.7
- New forms can be created, compounding etc.

One of the most common methods is **finite-state-machines**

w3L14: W3L3: Finite-state methods for morphology

Finite State Automaton (FSA)



What is FSA?

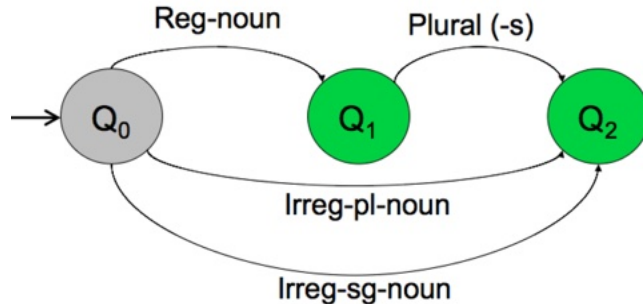
- A kind of directed graph
- Nodes are called states, edges are labeled with symbols (possibly empty ϵ)
- Start state and accepting states
- Recognizes regular languages, i.e., languages specified by regular expressions

Colr - input will not be accepted by the FSA - will not end up in accepting state

FSA for nominal inflection in English

Singular to plural - $Q_0 \rightarrow Q_1$: boy, cat will transition to plural by adding s

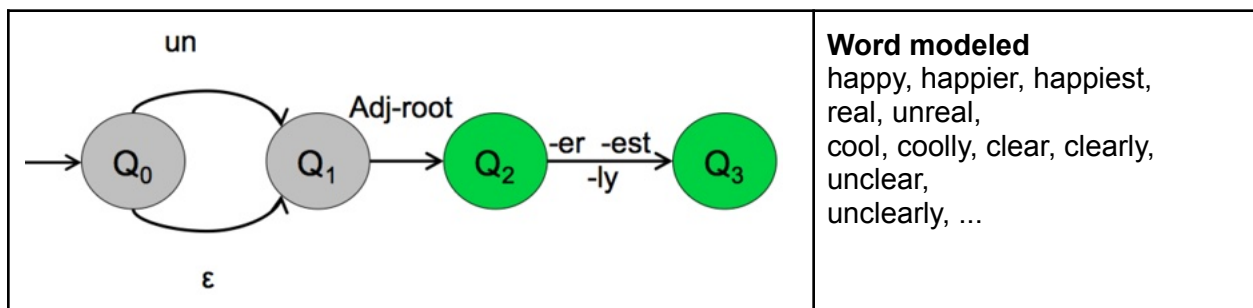
There will be some intermediate nodes, which are not shown



irregular goose $\rightarrow$ geese

FSA for English Adjectives

Prefix -un, Adj-root = happy

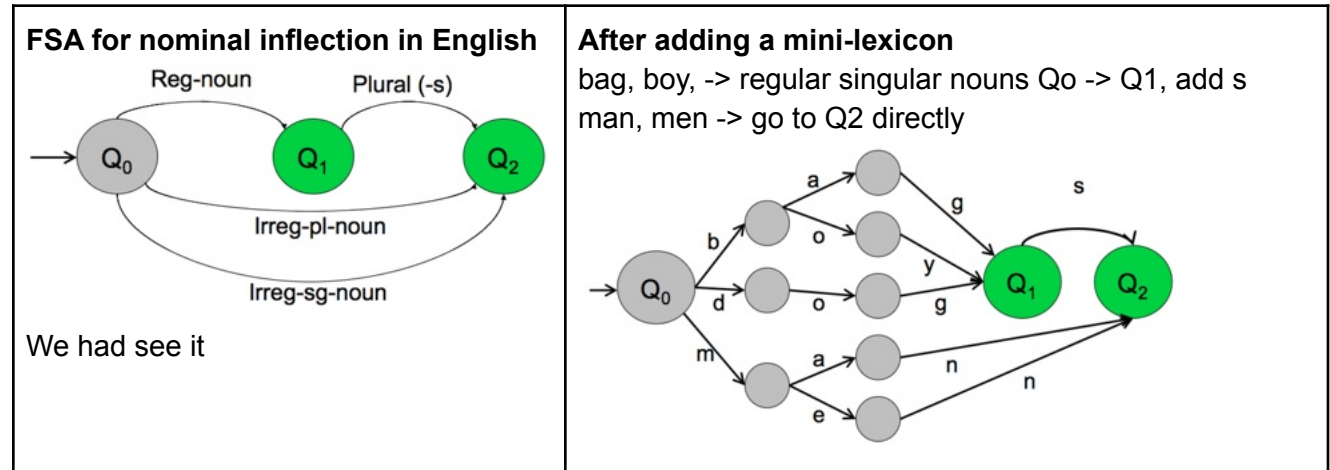


Morphotactics

The last two examples model some parts of the English morphotactics
But what about the information about regular and irregular roots?

Lexicon

Can we include the lexicon in the FSA?



Is this the goal with which we started?
It can recognize various words

Non-deterministic ND-FSA to deterministic FSA

Some properties of FSAs: Elegance

- Recognizing problem can be solved in **linear** time (independent of the size of the automaton)
 - Length of String
- There is an algorithm to transform each automaton into a unique equivalent automaton with the least number of states
- An FSA is **deterministic** iff it has no empty (ϵ) transition and for each state and each symbol, there is at most one applicable transition
- Every **non-deterministic automaton** can be transformed into a deterministic one

Not sufficient:

Lemma for word boy, can D-FA help? No

But ...

FSAs are language **recognizers/generators**.

We **need transducers** to build Morphological Analyzers

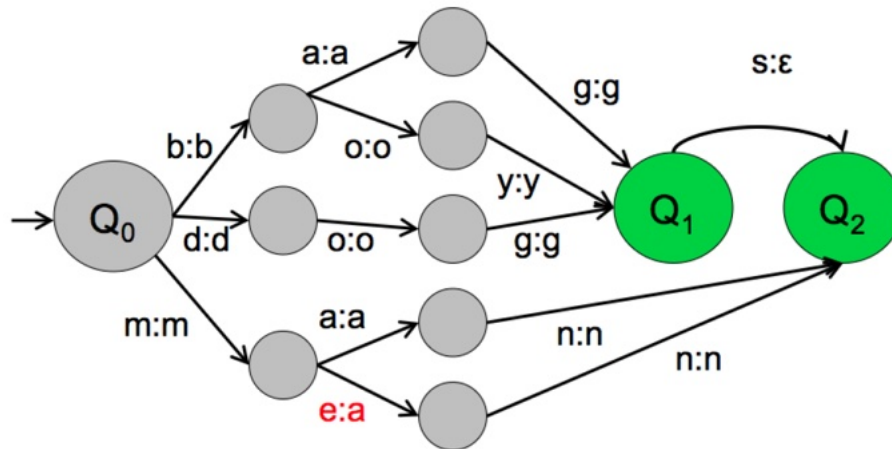
Finite State Transducers

Translate strings from one language to strings in another language

Like FSA, but each edge is associated with two strings

(Input char symbol to output char symbol - on arc) - Input men - output man

An example FST

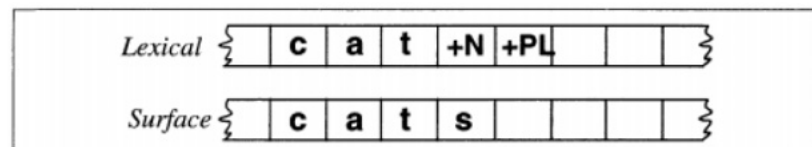


Two-level morphology

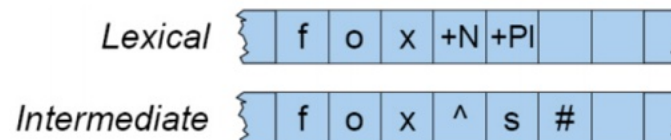
Given the input **cats**, we would like to output **cat+N+PL**, telling us that cat is a plural noun.

We do this via a version of two-level morphology,

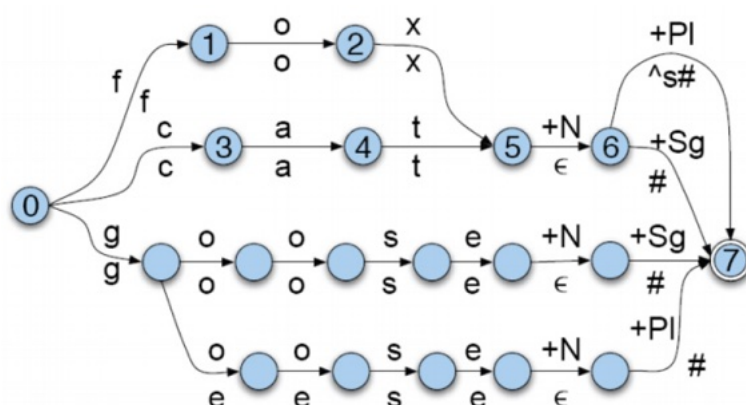
- a correspondence between a **lexical** level (morphemes and features) to a
- **surface** level (actual spelling).



Intermediate tape for Spelling change rules

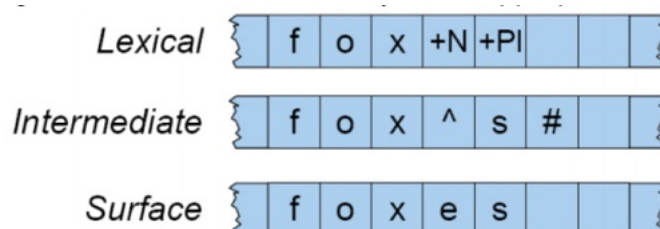


English Nominal Inflection FST (intermediate form)



Spelling Handling - Example:

A spelling change rule would insert an e only in the appropriate environment.



Transition will depend on ending character

Rule Handling

Rule Notation (Context sensitive grammar)

Between c and d, if a appears, replace it with b
 $a \rightarrow b \mid c_d$: "rewrite a as b when it occurs between c and d ."

$a \rightarrow b / c_d$
 $\underline{c} a \underline{d} \rightarrow \underline{c} b \underline{d}$
 Inter. $\rightarrow$ Surface

Morphological Analysis: Approaches

Two different ways to address phonological/graphemic variations

- **Linguistic approach:**
 - Stem is defined. Apply suffix
 - A phonological component accompanying the simple concatenative process of attaching an ending
- **Engineering approach:** Phonological changes and irregularities are factored into endings and a higher number of paradigms
 - No separate units for spell changes

Different Approaches: Example from Czech

	woman	owl	draft	iceberg	vapor	fly
S1	žen-a	sov-a	skic-a	kr-a	pár-a	mouch-a
S2	žen-y	sov-y	skic-i	kr-y	pár-y	mouch-y
S3	žen-ě	sov-ě	skic-e	kř-e	pář-e	mouš-e
:						
P2	žen-0	sov-0	skic-0	ker-0	par-0	much-0

A linguistic approach

$$\begin{matrix} \text{žen} + \begin{Bmatrix} a \\ y \\ \text{ě} \\ 0 \end{Bmatrix} & \text{sov} + \begin{Bmatrix} a \\ y \\ \text{ě} \\ 0 \end{Bmatrix} & \text{skic} + \begin{Bmatrix} a \\ y \\ \text{ě} \\ 0 \end{Bmatrix} & \text{kr} + \begin{Bmatrix} a \\ y \\ \text{ě} \\ 0 \end{Bmatrix} & \text{pár} + \begin{Bmatrix} a \\ y \\ \text{ě} \\ 0 \end{Bmatrix} & \text{mouch} + \begin{Bmatrix} a \\ y \\ \text{ě} \\ 0 \end{Bmatrix} \end{matrix}$$

An engineering approach

$$\begin{matrix} \text{žen} + \begin{Bmatrix} a \\ y \\ \text{ě} \\ 0 \end{Bmatrix} & \text{sov} + \begin{Bmatrix} a \\ y \\ \text{ě} \\ 0 \end{Bmatrix} & \text{skic} + \begin{Bmatrix} a \\ \text{i} \\ \text{e} \\ 0 \end{Bmatrix} & \text{k} + \begin{Bmatrix} \text{ra} \\ \text{ry} \\ \text{ře} \\ \text{er} \end{Bmatrix} & \text{p} + \begin{Bmatrix} \text{ára} \\ \text{áry} \\ \text{áře} \\ \text{ar} \end{Bmatrix} & \text{m} + \begin{Bmatrix} \text{oucha} \\ \text{ouchy} \\ \text{ouše} \\ \text{uch} \end{Bmatrix} \end{matrix}$$

How does on capture irregularities:

- Linguistic
- Engineering
 - In iceberg - take k as stem instead of kr
 - Handle spelling change rules there itself

Tools Available

AT&T FSM Library and Lextools

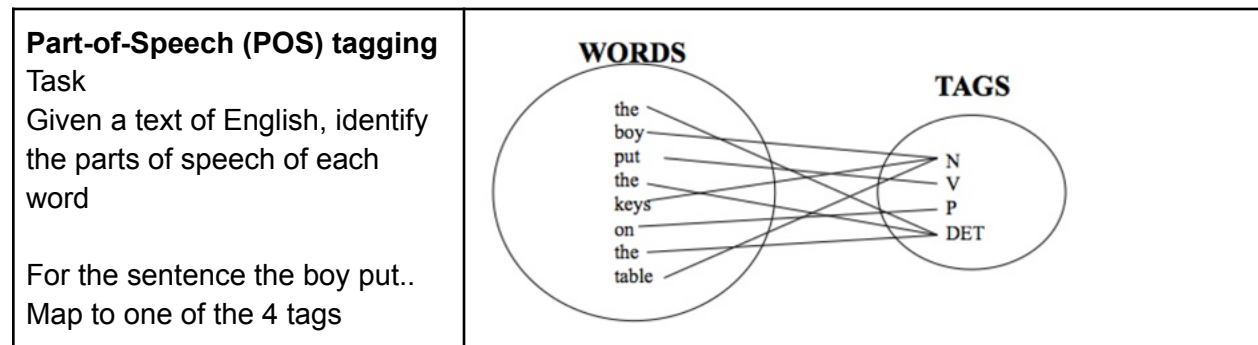
- <http://www2.research.att.com/~fsmtools/fsm/>

OpenFST (Google and NYU)

- <http://www.openfst.org/>

w3L15: L3W4: Introduction to POS Tagging

Disambiguation - what is the actual category for each word, give unique answer



Parts of Speech: How many?

Open class words (content words)

- **nouns, verbs, adjectives, adverbs**
- mostly content-bearing: they refer to **objects, actions**, and **features** in the world
- open class, since new words are **added** all the time
- google = search

Closed class words

- Fixed set
- **pronouns, determiners, prepositions, connectives, ...**
- there is a limited number of these
- *mostly functional*: to **tie the concepts** of a sentence together

POS examples (coarse grained category)

▪ N	noun	chair, bandwidth, pacing
▪ V	verb	study, debate, munch
▪ ADJ	adj	purple, tall, ridiculous
▪ ADV	adverb	unfortunately, slowly,
▪ P	preposition	of, by, to
▪ PRO	pronoun	I, me, mine
▪ DET	determiner	the, a, that, those

Fine grained:

- singular or plural noun
- Go to grammatical detail of the sentence
- Proper noun, past tense
-

POS tagging: Choosing a tagset

- To do POS tagging, a standard set needs to be chosen
- Could pick very coarse tagsets
 - N, V, Adj, Adv
- More commonly used set is finer grained, “**UPenn TreeBank tagset**”, **45 tags**

A Nice Tutorial on POS tags (what is the difference - particle vs determinant)

<https://sites.google.com/site/partofspeechhelp/>

UPenn TreeBank POS tag set

Tag	Description	Example	Tag	Description	Example
CC	Coordin. Conjunction	<i>and, but, or</i>	SYM	Symbol	<i>+, %, &</i>
CD	Cardinal number	<i>one, two, three</i>	TO	“to”	<i>to</i>
DT	Determiner	<i>a, the</i>	UH	Interjection	<i>ah, oops</i>
EX	Existential ‘there’	<i>there</i>	VB	Verb, base form	<i>eat</i>
FW	Foreign word	<i>mea culpa</i>	VBD	Verb, past tense	<i>ate</i>
IN	Preposition/sub-conj	<i>of, in, by</i>	VBG	Verb, gerund	<i>eating</i>
JJ	Adjective	<i>yellow</i>	VCN	Verb, past participle	<i>eaten</i>
JJR	Adj., comparative	<i>bigger</i>	VBP	Verb, non-3sg pres	<i>eat</i>
JJS	Adj., superlative	<i>wildest</i>	VBZ	Verb, 3sg pres	<i>eats</i>
LS	List item marker	<i>1, 2, One</i>	WDT	Wh-determiner	<i>which, that</i>
MD	Modal	<i>can, should</i>	WP	Wh-pronoun	<i>what, who</i>
NN	Noun, sing. or mass	<i>llama</i>	WP\$	Possessive wh-	<i>whose</i>
NNS	Noun, plural	<i>llamas</i>	WRB	Wh-adverb	<i>how, where</i>
NNP	Proper noun, singular	<i>IBM</i>	\$	Dollar sign	<i>\$</i>
NNPS	Proper noun, plural	<i>Carolinas</i>	#	Pound sign	<i>#</i>
PDT	Predeterminer	<i>all, both</i>	“	Left quote	<i>(‘ or “)</i>
POS	Possessive ending	<i>’s</i>	”	Right quote	<i>(’ or ”)</i>
PRP	Personal pronoun	<i>I, you, he</i>	(	Left parenthesis	<i>([, (, { , <)</i>
PRP\$	Possessive pronoun	<i>your, one’s</i>	)	Right parenthesis	<i>([,) , } , >)</i>
RB	Adverb	<i>quickly, never</i>	,	Comma	<i>,</i>
RBR	Adverb, comparative	<i>faster</i>	.	Sentence-final punc	<i>(. ! ?)</i>
RBS	Adverb, superlative	<i>fastest</i>	:	Mid-sentence punc	<i>(: ; ... - -)</i>
RP	Particle	<i>up, off</i>			

<http://textanalysisonline.com/spacy-pos-tagging>

With UPEN:

<https://textinspector.com/help/tagger/>

Muslims|NOUN **fast**|**VERB** during|ADP Ramadan|NOUN

She|NOUN spoke|VERB so|ADV **fast**|ADV that|ADP i|NOUN could|VERB not|ADV follow|VERB her|NOUN .|PUNCT

There|ADV are|VERB several|ADJ **fast**|ADJ food|NOUN centers|NOUN around|ADP the|DET city|NOUN .|PUNCT

He|NOUN was|VERB **fast**|ADV asleep|ADJ when|ADV i|NOUN reached|VERB home|NOUN .|PUNCT

Using the UPenn tagset

Example Sentence

- The grand jury commented on a number of other topics.

POS tagged sentence

- The/**DT** grand/**JJ** jury/**NN** commmented/**VBD** on/**IN** a/**DT** number/**NN** of/**IN** other/**JJ** topics/**NNS** ./.

Why is POS tagging hard?

Words often have **more than one POS**: **back** (*only by context you can define POS*)

- The back door: *back*/**JJ**
- On my back: *back*/**NN**
- Win the voters back: *back*/**RB**
- Promised to back the bill: *back*/**VB**

POS tagging problem

- To determine the POS tag for a particular instance of a word

How common the problem is: for 10 words - just use rules

10% is a real problem, will need a model

Toten - same word multiple times

11min:

Ambiguous word types in the Brown Corpus Ambiguity in the Brown corpus 40% of word tokens are ambiguous <i>(real problem)</i> 12% of word types are ambiguous Breakdown of ambiguous word types:	Unambiguous (1 tag)		35,340
	Ambiguous (2–7 tags)		4,100
	2 tags		3,760
	3 tags		264
	4 tags		61
	5 tags		12
	6 tags		2
	7 tags		1 ("still")

Getting 2-3 tags is common, especially two

How bad is the ambiguity problem?

- One tag is usually **more likely** than the others.
- In the Brown corpus, **race** is a **noun 98%** of the time, and a **verb 2%** of the time
 - Naively just assign as noun - what is simple baseline
- A tagger for English that simply chooses the most likely tag for each word can achieve good performance
- Any new approach should be compared against the unigram baseline (assigning each token to its most likely tag)

Deciding the correct POS

Can be difficult even for people

- Mrs./NNP Shaefer/NNP never/RB got/VBD **around/**_ to/TO joining/VBG.
- All/DT we/PRP gotta/VBN do/VB is/VBZ go/VB **around/**_ the/DT corner/NN.
- Chateau/NNP Petrus/NNP costs/VBZ around/_ 2500/CD.

Can be difficult even for people

- Mrs./NNP Shaefer/NNP never/RB got/VBD around/RP to/TO joining/VBG.
- All/DT we/PRP gotta/VBN do/VB is/VBZ go/VB around/IN the/DT corner/NN.
- Chateau/NNP Petrus/NNP costs/VBZ around/RB 2500/CD.

Relevant knowledge for POS tagging

The word itself

- Some words may only be nouns, e.g. **arrow**
- Some words are ambiguous, e.g. **like, flies**
- **Probabilities** may help, if one tag is more likely than another

Local context

- **Two determiners rarely** follow each other
- **Two base form verbs rarely** follow each other
- **Determiner** is almost always followed by **adjective** or **noun**

POS tagging: Two approaches

Rule-based Approach

- Assign each word in the input a list of potential POS tags
- Then winnow down this list to a single tag using hand-written rules

Statistical tagging

- Get a **training** corpus of **tagged** text, learn the transformation rules from the most frequent tags (TBL tagger - earlier model - can i learn transformation rule)
- **Probabilistic**: Find the most likely sequence of tags **T** for a sequence of words **W**

TBL Tagger (Idea)

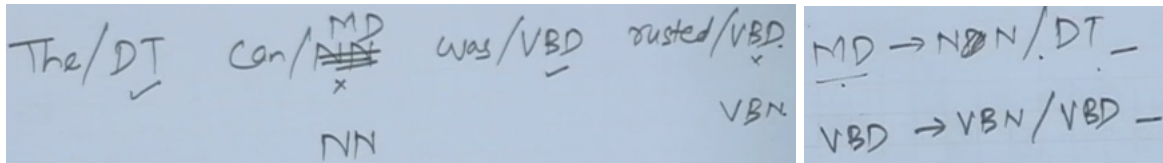
Label the training set with most frequent tags

- **Sentence:** The can was rusted.
- **Know tag also:** The/DT can/MD was/VBD rusted/VBD.

Add transformation rules to reduce training mistakes

Need to write rule so that it can be transformed:

- MD -> NN: DT_ (if previous is Determiner change to MD)
- VBD -> VBN: VBD_



Probabilistic Tagging: Two different families of models

Problem at hand

- We have some data $\{ (d, c) \}$ of paired **observations d** and hidden **classes c**.

Different instances of d and c

- **Part-of-Speech Tagging: words are observed and tags are hidden.**
- **Text Classification: sentences/documents are observed and the category is hidden.**
 - Categories can be positive/negative/neutral for **sentiments** ..
 - sports/politics/business for documents ...

What gives rise to the two families?

Whether they generate the observed data from hidden stuff or the hidden structure given the data?

Generative vs. Conditional Models

Generative (Joint) Models

- *Generate data from class [to generate doc - think about topic-class-politics -> write words, find probability]*

Generate the observed data from hidden stuff, i.e. put a probability over the observations given the class: $P(d, c)$ in terms of $P(d | c)$

e.g. Naïve Bayes' classifiers, Hidden Markov Models etc.

Discriminative (Conditional) Models

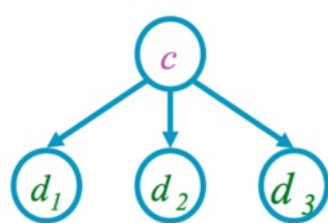
- *Hidden state is generated from data (given obs directly find out the probability of class)*

Take the data as given, and put a probability over hidden structure given the data: $P(c|d)$

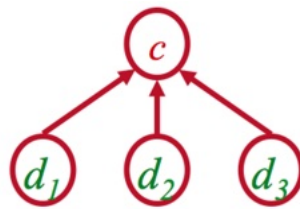
e.g. Logistic regression, maximum entropy models, conditional random fields

SVMs, perceptron, etc. are discriminative classifiers but not directly probabilistic

Generative vs. Discriminative Models



Naive Bayes



Logistic Regression

class to doc. Doc to class

Joint vs. conditional likelihood

- A joint model gives probabilities $P(d, c)$ and tries to maximize this joint likelihood.
- A conditional model gives probabilities $P(c | d)$, taking the data as given and modeling only the conditional probability of the class.

w3L16: W3L5: Hidden Markov Models for POS Tagging

Given sentence -> what are unique POS for each word

Probabilistic Tagging

- $W = w_1 \dots w_n$ - words in the corpus (observed)
 - $T = t_1 \dots t_n$ - the corresponding tags (unknown)
- t can take any of 45 tags

Tagging: Probabilistic View (Generative Model)

- find tag sequences -> find one which has max probability

Need to inverse ($T|W$) as it is generative write in form T class comes then W
 $P(W|T)$ using Bayes Theorem

$$\begin{aligned}\hat{T} &= \operatorname{argmax}_T P(T|W) \\ &= \operatorname{argmax}_T \frac{P(W|T)P(T)}{P(W)} \\ &= \operatorname{argmax}_T P(W|T)P(T) \\ &= \operatorname{argmax}_T \prod_i P(w_i|w_1 \dots w_{i-1}, t_1 \dots t_i)P(t_i|t_1 \dots t_{i-1})\end{aligned}$$

Find

© CET
I.I.T. KGP

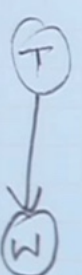
$$\hat{T} = \underset{T}{\operatorname{argmax}} P(T|W)$$

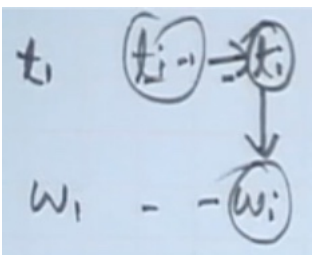
$$= \underset{T}{\operatorname{argmax}} P(t_1 \dots t_n | w_1 \dots w_n)$$

$$\rightarrow \underset{T}{\operatorname{argmax}} \frac{P(W|T) P(T)}{P(W)}$$

$$= \underset{T}{\operatorname{argmax}} \underline{P(W|T) P(T)}$$

$$= P(w_1 \dots w_n | t_1 \dots t_n) P(t_1 \dots t_n)$$

$$= \prod_{i=1}^n P(\underline{w_i} | w_1 \dots w_{i-1}, t_1 \dots t_n) P(t_i | t_1 \dots t_{i-1})$$




$$= \prod_{i=1}^n P(\underline{w_i} | w_1 \dots w_{i-1}, t_1 \dots t_n) P(t_i | t_1 \dots t_{i-1})$$

$P(w_i t_i)$	$P(t_i t_{i-1})$
	[bigram assumption]

Further simplifications

$$\hat{T} = \operatorname{argmax}_T \prod_i P(w_i | w_1 \dots w_{i-1}, t_1 \dots t_i) P(t_i | t_1 \dots t_{i-1})$$

- The probability of a word appearing depends only on its own POS tag
 $P(w_i | w_1 \dots w_{i-1}, t_1 \dots t_i) \approx P(w_i | t_i)$
- Bigram assumption: the probability of a tag appearing depends only on the previous tag

$$P(t_i | t_1 \dots t_{i-1}) \approx P(t_i | t_{i-1})$$

- Using these simplifications:

$$\hat{T} = \operatorname{argmax}_T \prod_i P(w_i | t_i) P(t_i | t_{i-1})$$

Computing the probability values [have manually annotated data]

Tag Transition probabilities $p(t_i | t_{i-1})$

$$P(t_i | t_{i-1}) = \frac{C(t_{i-1}, t_i)}{C(t_{i-1})}$$

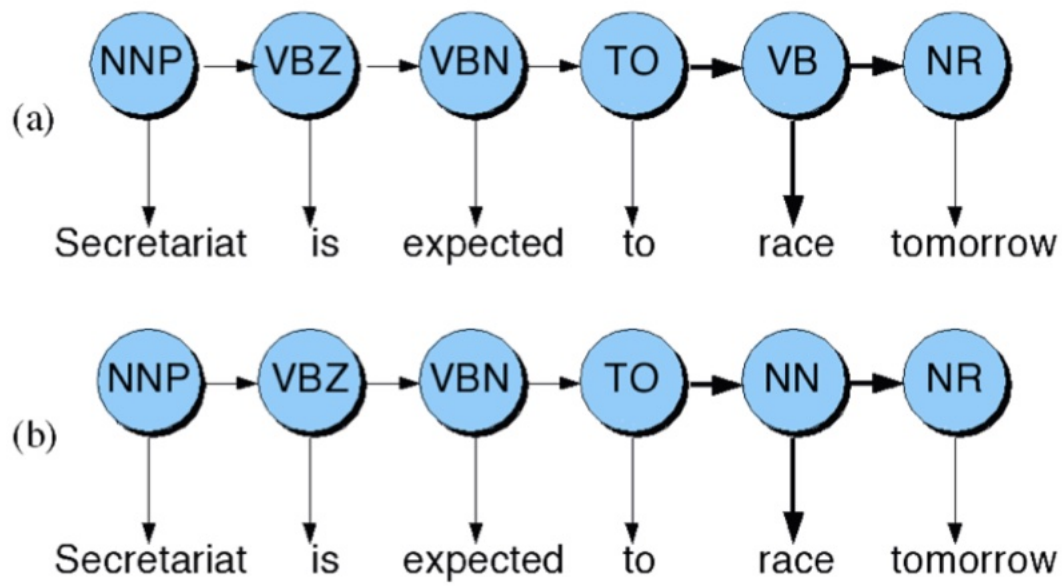
$$P(NN | DT) = \frac{C(DT, NN)}{C(DT)} = \frac{56,509}{116,454} = 0.49$$

Word Likelihood probabilities $p(w_i | t_i)$

$$P(w_i | t_i) = \frac{C(t_i, w_i)}{C(t_i)}$$

$$P(is | VBZ) = \frac{C(VBZ, is)}{C(VBZ)} = \frac{10,073}{21,627} = 0.47$$

Disambiguating “race” - for some ambiguous model - not complete model



Difference in probability due to - which every multiplication give higher values
 Interpretation 1 and interpretation 2

Inter. 1	Inter. 2
$P(VB TO) \times$	$P(NN TO) \times$
$P(NR VB) \times$	$P(NR NN) \times$
$P(race VB)$	$P(race NN)$
= ✓	= ✓

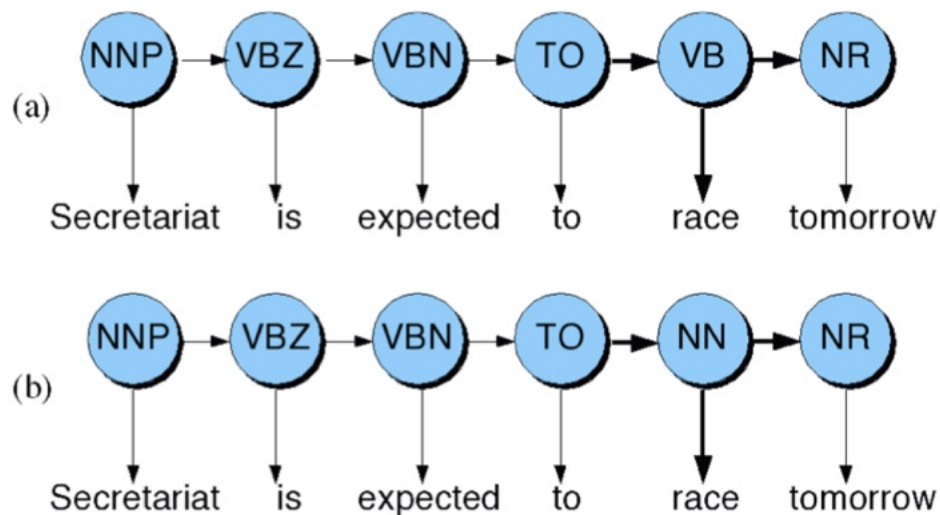
- $P(VB|TO)$ vs. $P(NN|TO)$
- $P(race|VB)$ vs. $P(race|NN)$
- $P(NR|VB)$ vs. $P(NR|NN)$

After computing the probabilities

- $P(NN|TO)P(NR|NN)P(race|NN) = 0.0047 \times 0.0012 \times 0.00057 = 0.00000000032$
- $P(VB|TO)P(NR|VB)P(race|VB) = 0.83 \times 0.0027 \times 0.00012 = 0.000000027$

The race word should be Verb not as a Noun

What is this model? - This is a Hidden Markov Model



Hidden Markov Models

- Tag Transition probabilities $p(t_i|t_{i-1})$
- Word Likelihood probabilities (emissions) $p(w_i|t_i)$

Tag prob: One tag to other tag

Word is observation: So emission probability:

- What we have described with these probabilities is a hidden markov model.
- Let us quickly introduce the Markov Chain, or observable Markov Model.

Markov Chain = First-order Markov Model

States are observation - weather: 3 states - sunny rainy, foggy

Weather example

- Three types of weather: *sunny, rainy, foggy*
- q_n : variable denoting the weather on the n^{th} day
- We want to find the following conditional probabilities:

$$P(q_n | q_{n-1}, q_{n-2}, \dots, q_1)$$

First-order Markov Assumption

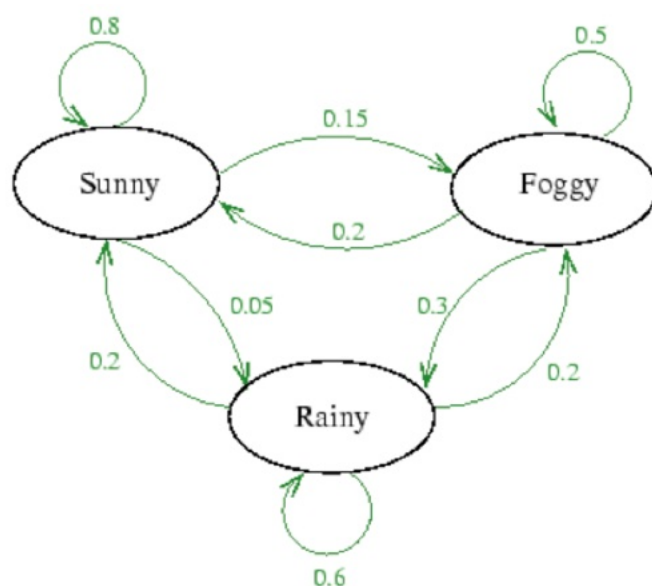
$$P(q_n | q_{n-1}, q_{n-2}, \dots, q_1) = P(q_n | q_{n-1})$$

1st order: q-n depends only on previous day!

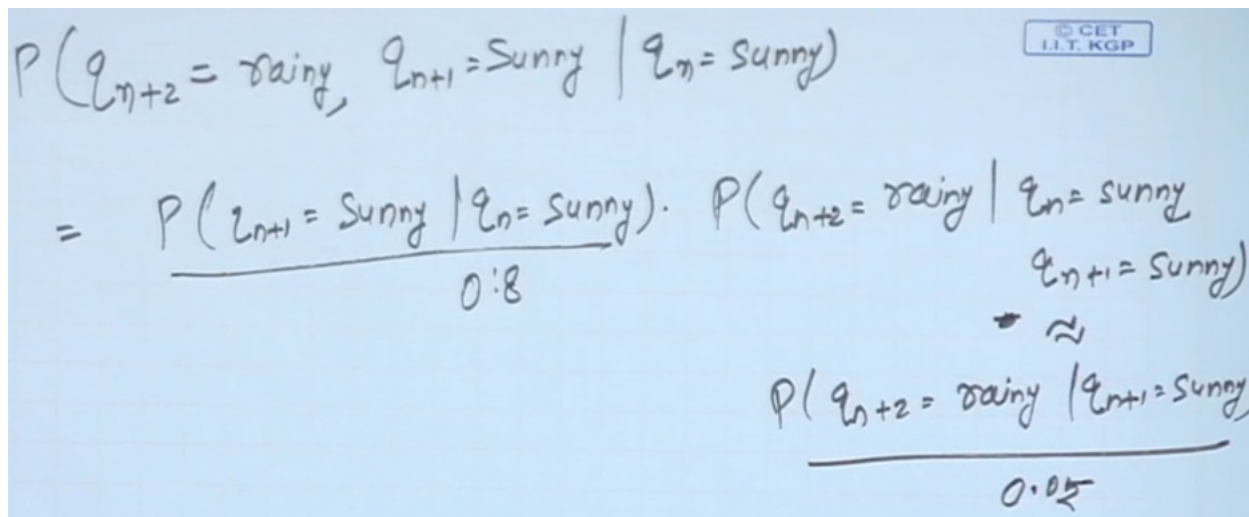
16min: Markov Chain Transition Table

Table 1: Probabilities $P(q_{n+1} | q_n)$ of tomorrow's weather based on today's weather

Today's weather	Tomorrow's weather		
			
	0.8	0.05	0.15
	0.2	0.6	0.2
	0.2	0.3	0.5



Using Markov Chain: Given that today the weather is sunny, what is the probability that tomorrow is sunny and the day after is rainy?



Handwritten calculation on a blue background with a stamp in the top right corner that reads "© CET I.I.T. KGP". The calculation is as follows:

$$P(q_{n+2} = \text{rainy}, q_{n+1} = \text{sunny} \mid q_n = \text{sunny})$$

$$= \frac{P(q_{n+1} = \text{sunny} \mid q_n = \text{sunny}) \cdot P(q_{n+2} = \text{rainy} \mid q_{n+1} = \text{sunny})}{0.8}$$

The second term in the numerator is simplified to 0.05, resulting in:

$$= \frac{0.05}{0.8} = 0.04$$

$$P(q_2 = \text{sunny}, q_3 = \text{rainy} \mid q_1 = \text{sunny})$$

$$\begin{aligned}
 &= P(q_3 = \text{rainy} \mid q_2 = \text{sunny}, q_1 = \text{sunny}) \times P(q_2 = \text{sunny} \mid q_1 = \text{sunny}) \\
 &= P(q_3 = \text{rainy} \mid q_2 = \text{sunny}) \times P(q_2 = \text{sunny} \mid q_1 = \text{sunny}) \\
 &= 0.05 \times 0.8 \\
 &= 0.04
 \end{aligned}$$

Markov Model:

States - you are observing

Transition are among state

Observations - are state

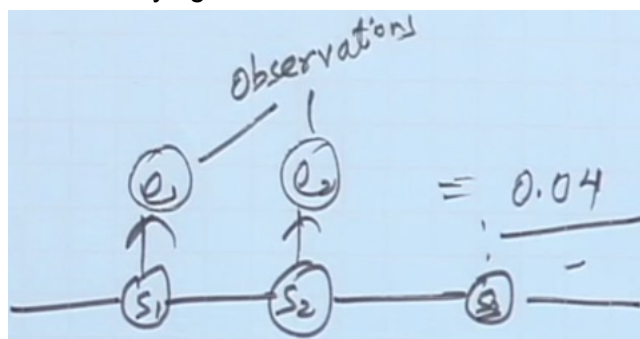
Hidden Markov model:

In text we observe word

States are **hidden** variable -> you can

Emissions are observation

Find underlying states



Hidden Markov Model (extension of markov chain - o/p symbols are diff than the states)

For Markov chains, the **output symbols** are the **same** as the **states** 'sunny' weather is both **observable** and **state**

But in POS tagging

- The **output** symbols are words
- But the **hidden** states are **POS tags**

A Hidden Markov Model is an extension of a Markov chain in which the output symbols are not the same as the states

We don't know which state we are in, until we use our model

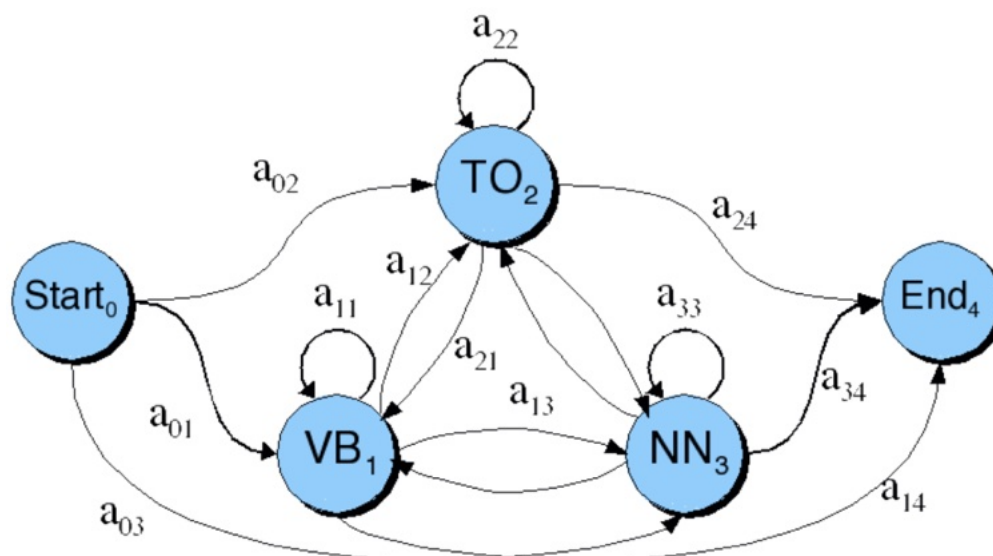
21min: Elements of an HMM model

- A set of states (here: the tags)
- An output alphabet (here: words)
- Initial state (here: beginning of sentence)
- State transition probabilities (here $p(t_n|t_{n-1})$)
- Symbol emission probabilities (here $p(w_i|t_i)$)

Graphical Representation

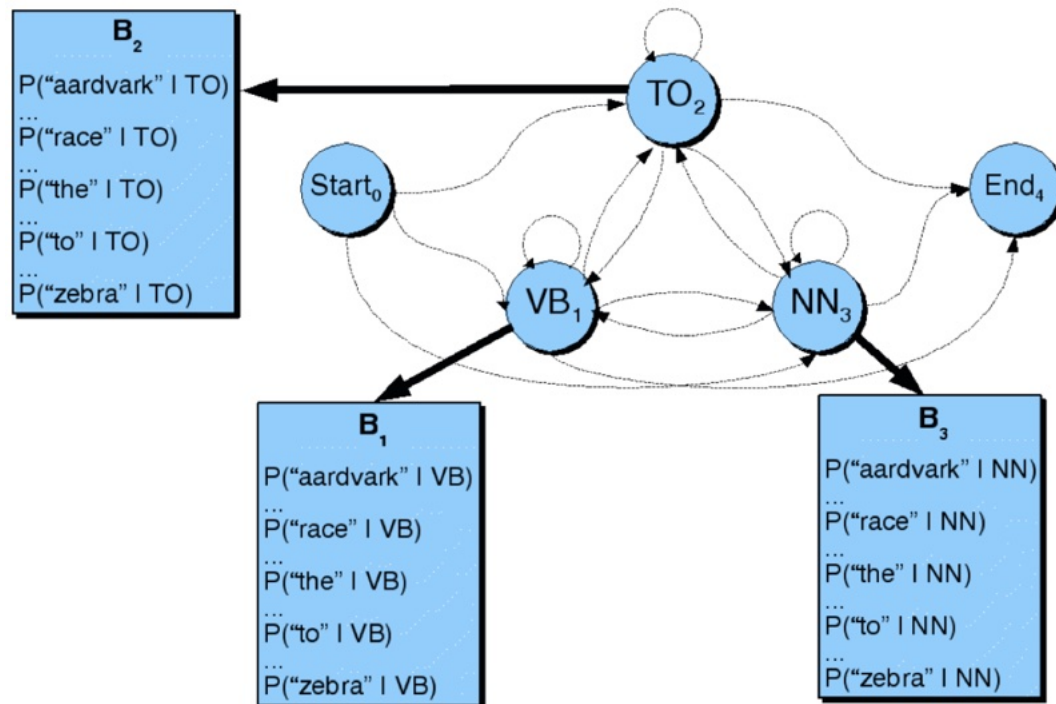
When tagging a sentence, we are walking through the state graph:

Edges are labeled with the state transition probabilities: $p(t_n|t_{n-1})$



Graphical Representation

At each state we emit a word: $P(w_n|t_n)$



Walking through the states: best path

At run time given a sentence, what tag should be used?

What is the sequence of tags which will be used

What tags a word can take

